

Question:

Develop a "**Library Management System**" to demonstrate Object-Oriented Programming concepts.

Steps

1. Create a **Book** class with **bookId**, **bookName**, **author**, and **price**.
2. Create a default constructor and a parameterized constructor in the **Book** class.
3. Create getter and setter methods to implement encapsulation.
4. Create a **displayBook()** method to display book details.
5. Create a **Person** class with **name** and **age**.
6. Create **Student** and **Faculty** classes that inherit from **Person**.
7. Use the **super** keyword to initialize the parent class values.
8. Create an **Area** class with overloaded **calculateArea()** methods for square, rectangle, and circle.
9. Create a **Vehicle** class with a **display()** method.
10. Create **Car** and **Bike** classes that override the **display()** method.

11. Create an abstract **Shape** class with an abstract **draw()** method.
12. Create **Circle** and **Rectangle** classes that implement the **draw()** method.
13. Create a **Printable** interface with a **print()** method.
14. Create a **Report** class that implements the **Printable** interface.
15. In the **main()** method, create objects and demonstrate each concept.
16. Display the results of classes, constructors, encapsulation, inheritance, method overloading, method overriding, abstraction, and interfaces.
17. End the program.

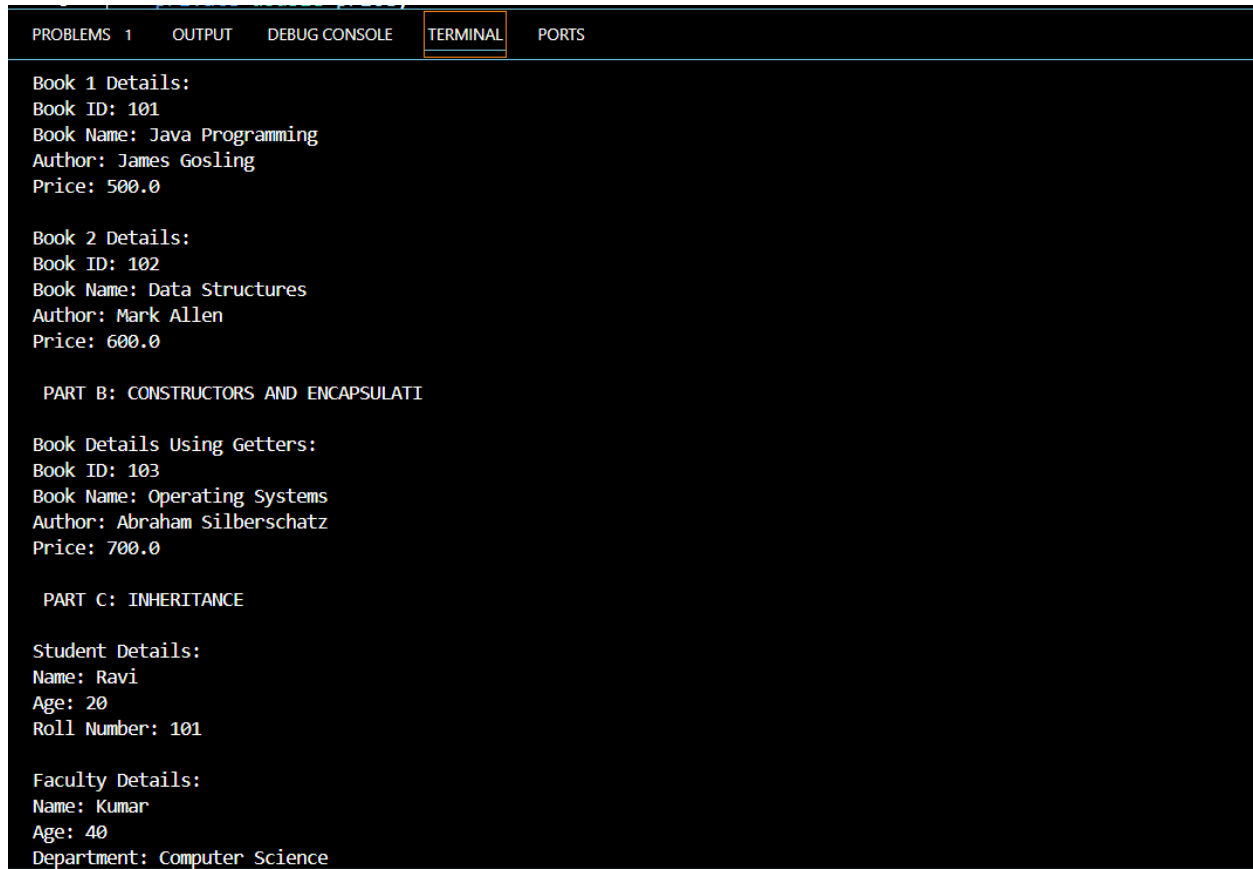
Algorithm

1. **Start**
2. Define the **Book** class with four private data members.
3. Initialize book details using constructors.
4. Access and modify book data using getter and setter methods.
5. Display the book details.
6. Define the **Person** class with **name** and **age**.
7. Derive **Student** and **Faculty** from **Person**.

8. Initialize inherited data using `super()`.
9. Display student and faculty details.
10. Define the `Area` class with overloaded `calculateArea()` methods.
11. Calculate the area of:
 - Square
 - Rectangle
 - Circle
12. Define the `Vehicle` class with `display()`.
13. Override `display()` in `Car` and `Bike`.
14. Create `Vehicle` references pointing to `Car` and `Bike` objects.
15. Call `display()` to demonstrate method overriding.
16. Define the abstract `Shape` class with `draw()`.
17. Create `Circle` and `Rectangle` classes extending `Shape`.
18. Create objects of `Circle` and `Rectangle` using `Shape` references.
19. Call `draw()` to demonstrate abstraction.
20. Define the `Printable` interface with `print()`.
21. Implement the interface in the `Report` class.
22. Create a `Printable` reference pointing to a `Report` object.

23. Call `print()` to demonstrate interface implementation.

24. **Stop.**

A screenshot of an IDE's terminal window. The terminal has a dark background with light-colored text. At the top, there is a tab bar with five tabs: 'PROBLEMS 1', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and highlighted with a yellow border), and 'PORTS'. The terminal displays the following output:

```
Book 1 Details:
Book ID: 101
Book Name: Java Programming
Author: James Gosling
Price: 500.0

Book 2 Details:
Book ID: 102
Book Name: Data Structures
Author: Mark Allen
Price: 600.0

PART B: CONSTRUCTORS AND ENCAPSULATI

Book Details Using Getters:
Book ID: 103
Book Name: Operating Systems
Author: Abraham Silberschatz
Price: 700.0

PART C: INHERITANCE

Student Details:
Name: Ravi
Age: 20
Roll Number: 101

Faculty Details:
Name: Kumar
Age: 40
Department: Computer Science
```